

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 3

NEURAL NETWORKS

Reading Time	20 minutes
Prior Knowledge	Linear and Non-linear data, gradient
Keywords	Activation function, Loss function, Neuron, Layers

In the previous lesson, we covered support vector machines, which can handle both linear and non-linear data provided the appropriate kernel is used. In this lesson, we will cover a neural network predictor that is effective for both linear and non-linear data.

1. Neural Network Basics

Neural networks (NNs) are often described as a way to replicate the human brain with a computer. When we think about the process of identifying an image, such as an image of a dog, we first view the image with our eyes thereby sending inputs to the brain. The next step would be the brain analyzing the image based on what we've seen about dogs in the past. Finally, the brain has made a decision/conclusion that this is an image of a dog. To summarize this process, we could have:

Input -> analyze -> decision.

This task is quite simple for us; however, to get a computer to perform the same image recognition task for a dog is not trivial. The process is similar in that the computer undertakes a similar process:

Input -> process/calculations -> output.

Let's look at an example to understand the process of a neural network to discover the relationship between inputs and outputs. We'll consider the simple problem of converting mass from pounds (lbs) to kilograms (kg). Suppose we didn't know the formula for the conversion, and all we had to go on was the observation:

kg	lbs
50	110.23

We make a critical assumption here: that the relationship between kg and lbs is linear. I know it's a good guess since we know the formula is $\text{lbs} = 2.2046 \times \text{kg}$, but often, we don't know the relationship. So we've made our guess that the relationship is $\text{lbs} = c \times \text{kg}$, where c is the slope. This linear equation is what is called the *activation function* in neural networks. We begin by making a guess about $c=1.5$, and with this guess, our lbs estimate is $\text{lbs} = 1.5(50) = 75$ lbs. we therefore seem to have underestimated by $110.23 - 75 = 35.23$ lbs. Okay, so we guess again, going higher for c based on the feedback from our first guess, and go with $c=3$, resulting in our second estimate of $50(3) = 150$ lbs. It seems we overestimated this time by 39.77 lbs. We therefore need to go with a value of $1.5 < c < 3$. We do this iteratively until we get to a value of $c=2.2$, which results in 110 lbs. Pretty close. We can decide this is close enough, therefore settling on $c=2.2$. We can summarize the iterations below,

Table 1: Summary of Iterations to Find Optimal Slope c .

iteration	c	lbs estimate	(Actual-predicted)
1	1.5	75	35.23
2	3.0	150	-39.77
3	2.5	125	-14.77
4	2.2	110	0.23

The last column provided insight into how far away our guess was from the actual. This is an example of a *loss function* in a NN. The magnitude of the difference between the actual and estimate signals how we need to adjust our next guess for c . Think of the difference being large resulting in our next guess of c being largely different from the previous one. This type of feedback is what we call the *optimizer* in a NN. With this example, we more or less get the workings or flow of a NN. This can be summarized in figure 1 below. You'll notice a 'weight' component in the diagram, which we will get to in the next section. To do this, we will look at the architecture of the NN.

Figure 1: Flow Diagram of Neural Network Operation

2. Network Architecture

A neural network's architecture is shown in figure 2. The hidden layer is where the processing takes place, and do keep in mind that we can have more than one hidden layer, but for now, we keep it simple.

Figure 2: Architecture of Basic Neural Network

If you look at the connections (edges) between the input layer and hidden layer, you will notice all red circles (input nodes) are connected to all the blue circles (Hidden layer nodes). We call this type of connection a *dense* layer. Think of each input node as a feature feeding information into the NN, for example in a credit risk problem, one input node could be Age with another being Salary. Hence, the number of nodes in the input layer is the number of features or predictors. The number of nodes in the output layer will depend on the target variable. If the target were a binary class variable, then one node would be sufficient in the output layer since it returns the probability of being in the target class, e.g., probability of defaulting on a loan. For a regression-type problem, like the lbs to kg conversion discussed earlier, the output node would return the continuous value of the target variable, i.e., lbs.

3. Forward and Backward Propagation

NNs achieve predictions via a process called propagation. At each forward step, there are matrix multiplications taking place. Computers are relatively quick at doing these calculations, and it is worth understanding the process behind it. To begin, we must first identify the dimension of inputs at the start, i.e., at the input layer. We mentioned earlier that the number of nodes at the input layer corresponds to the number of predictors or features. This is the number of columns of the input matrix, and the number of rows corresponds to the number of observations, N say. We thus have a $N \times 3$ matrix of inputs for figure 2. In general, let's denote the number of features as p so figure 2 would have $p=3$. We shall denote the layers as follows:

1. Input layer: X_0
2. Hidden layer: X_1
3. Output layer: Y

We'll touch on some linear algebra here but nothing too intense other than matrix multiplication. In the forward propagation step moving from X_0 to X_1 , we have the following calculations taking place, which can be represented as matrix multiplication,

$$X_1 = f(X_0 w_0 + b_0), \tag{1}$$
 where w_0 is a matrix of $p \times 4$ *weights* since the hidden layer has 4 nodes, and f is a user specified activation function. The term b is a bias term that enables the activation function to shift, which allows more flexibility of the algorithm to fit the data. The activation function, in addition to specifying the linearity/non-linearity of data, also decides on which node to activate by controlling the value of the weights. For instance, suppose Age was not a strong predictor of defaulting on a loan, then the weight for that node would be small. To sequentially move to the output layer, we then have the following calculation,

$$Y = f(X_1 w_1 + b_1) = f(f(X_0 w_0 + b_0) w_1 + b_1), \tag{2}$$

by substituting (1) for X_1 . The dimension of w_1 will be 4×2 since we have 4 nodes in the hidden layer and 2 nodes in the output layer. This sequential process of matrix multiplication results in the following dimension changes:

- $X_0 \times w_0$: $(N \times p) \times (p \times 4) = N \times 4$ (Think of p canceling each other out)
- $X_1 \times w_1$: $(N \times 4) \times (4 \times 2) = N \times 2$

The dimensions of the weights and layers is necessary when specifying the architecture of the NN in a programming language, such as Python. Let's look at two videos that will help illustrate the inputs and outputs at the hidden nodes. Inputs of the hidden nodes can be shown in the video below.

```
In [ ]: from IPython.display import VimeoVideo  
  
VimeoVideo("782624672", h="60b0362998", width=600)
```

[Access Video Transcript here](#)

Outputs of hidden nodes can be show in this video:

```
In [ ]: VimeoVideo("782999865", h="2294f6c7e5", width=600)
```

[Access Video Transcript here](#)

Thus far, we have shown the forward propagation process to achieve outputs; however, it is the backward propagation process that enables the NN to learn and improve. This is the feedback process from the loss function to the optimizer, which eventually tells the algorithm how the weights should be adjusted. Ultimately, it is the weights, and some bias term, that will be adjusted to achieve optimal performance. A great benefit of NNs is that we do not have to restrict ourselves to just one hidden layer, and by increasing the layers, we increase our chances of solving non-linear data relationships. There is a problem that NNs face with this increase in layers, which is called the *vanishing gradient problem*.

Refer to the video below on how the bias is optimized using derivatives.

```
In [ ]: VimeoVideo("785293805", h="a725197abc", width=600)
```

[Access Video Transcript here](#)

4. Vanishing Gradient

We've briefly mentioned the role of the activation function in section 1. There are various activation functions to choose from with each having their benefits. The reader is directed to (Nwankpa 7) for a brief list of these functions. The sigmoid activation function is a popular function used in binary classification problems. It squashes the output into a small range between 0 and 1. The functional form may look familiar to you if you've worked with a logistic regression model as it is calculated as follows,

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

What this tells us is that for a large change in x or input, we have a small change in output. But we know that the gradient is given by,

$$\text{Gradient} = \frac{\text{change in output}}{\text{change in input}}$$

hence this implies the gradient tends to a small value for the sigmoid function. The connection of the sigmoid function to the loss function, L say, is that we know our output depends on the activation function. The more layers we have in our network, the more activation functions are required in the process. The idea of the backpropagation from table 1 was to minimize L or the difference in the last column through feedback. Calculus is very handy when we want to minimize some function f with respect to its input x . The process entails determining the value of x , say x' , that makes the derivative or gradient of $f = 0$. Backpropagation thus entails finding the derivative of the initial layer by multiplying the derivatives of each layer moving from the final to the initial layer, a process known as the chain rule. Thus, the problem arises where this product tends to zero with many layers, especially with the sigmoid activation function. This prevents optimal updating of the weights and bias.

Improvements to NNs have been made since 2009 to overcome this problem such as better activation functions like ReLu, better weight initialization schemes, and improved optimization schemes. NNs have come a long way since their inception, and with GPUs available, the speed of training these algorithms has been greatly improved.

5. Conclusion

In this lesson, we briefly explained the process of a NN. We mentioned its components such as inputs, hidden and output layers consisting of nodes, the forward and backward propagation process, and the role of the activation, loss, and optimizer in the operation of the algorithm. Neural networks also face problems such as vanishing gradients and overfitting if not tuned correctly. We'll look at an example of this in the next lesson where we train a neural network on a financial problem.

References

Nwankpa, Chigozie. "Activation Functions: Comparison of Trends in Practice and Research for Deep Learning." *arXiv Preprint arXiv*, 1811.03378, 2018, pp. 5–8, arxiv.org/pdf/1811.03378.pdf.